МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
„КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ”
НАВЧАЛЬНО-НАУКОВИЙ КОМПЛЕКС
„ІНСТИТУТ ПРИКЛАДНОГО СИСТЕМНОГО АНАЛІЗУ”

Лабораторна робота №5

з курсу: «Технології побудови розподілених баз даних та знань»

Виконав:
студент групи КН-51мп
Сидько Н.С.

КИЇВ
2026

Завдання:

1. Встановимо API Fuseki.

2. Програмно, використовуючи API Fuseki зчитаємо локальний triplestore №1, зчитаємо Інтернет-endpoint triplestore № 2, а також з'єднаємо локальні дані і Інтернет дані.

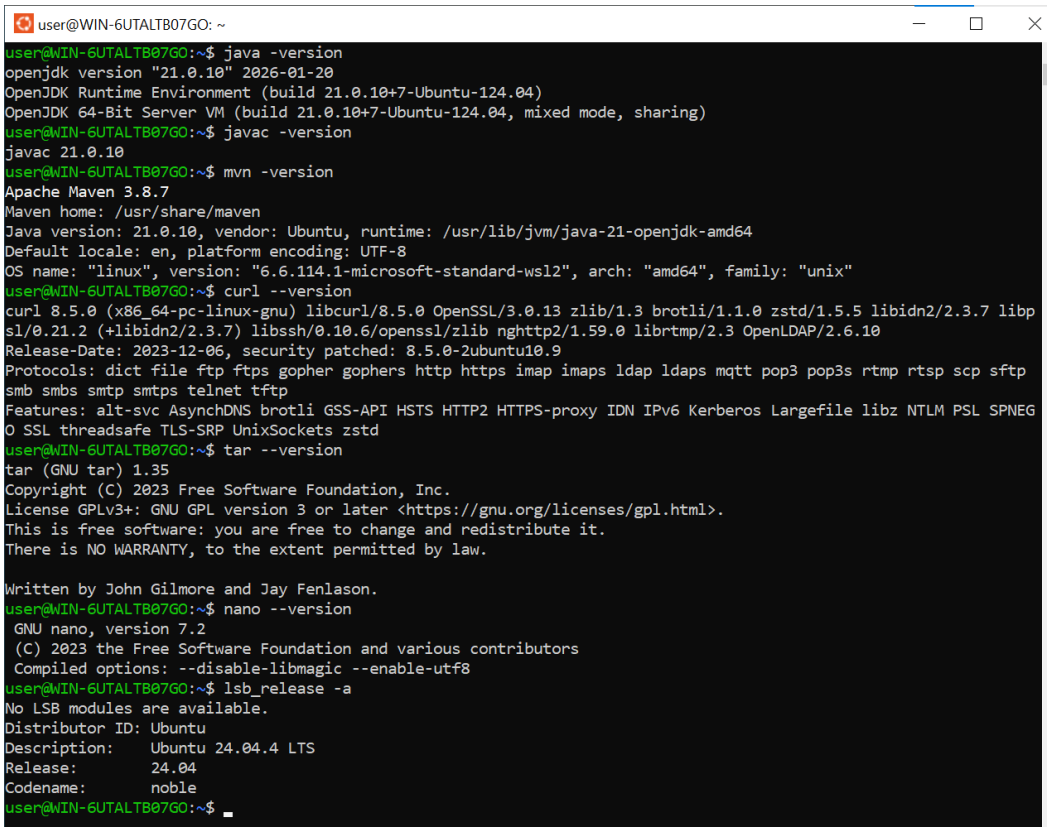
3. Програмно обчислимо тривалість виклику трьох запитів.

Хід роботи

1. Підготовка робочого середовища Ubuntu Linux WSL

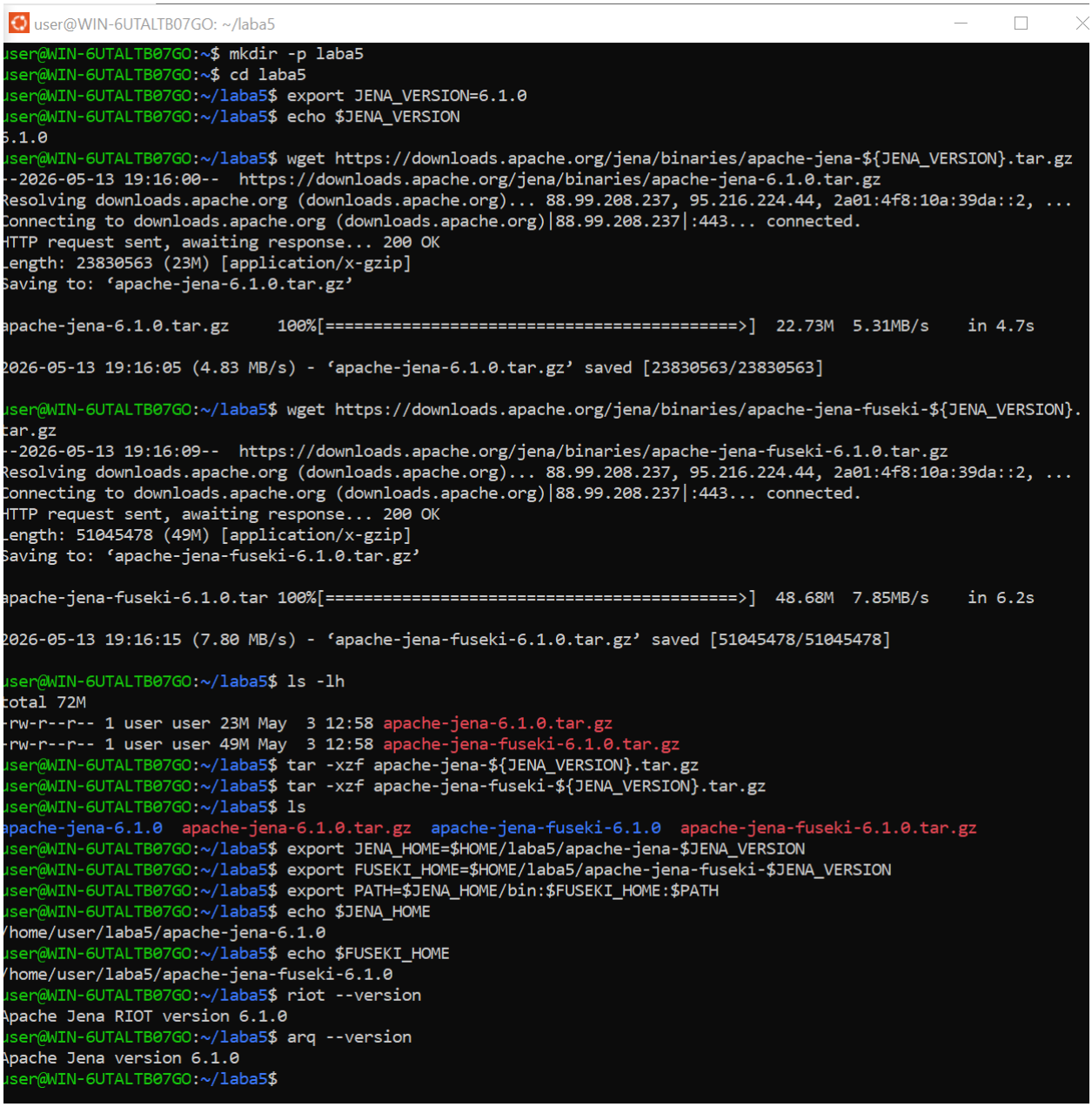
Для запуску Apache Jena Fuseki та створення клієнтської програми було встановлено Java JDK 21. Після встановлення було виконано перевірку версій java та javac, що підтвердило коректне встановлення Java-середовища. Це є необхідною умовою для подальшої роботи з Apache Jena та виконання програмної частини лабораторної роботи.

Також було встановлено Apache Maven. Цей інструмент використовується для збирання Java-проєктів та підключення зовнішніх бібліотек. У межах лабораторної роботи Maven буде потрібен для створення програмної частини, яка виконає три SPARQL-запити та виміряє час їх виконання.



2. Створення робочої директорії

На цьому етапі було створено робочу директорію laba5, у яку завантажено Apache Jena та Apache Jena Fuseki версії 6.1.0. Після завантаження архіви було розпаковано, а для зручності роботи задано змінні середовища JENA_HOME та FUSEKI_HOME. Перевірка команд riot --version і arq --version підтвердила, що інструменти Apache Jena встановлено та підключено коректно. Це дозволяє перейти до запуску Apache Jena Fuseki як локального SPARQL-сервера для створення RDF triplestore.

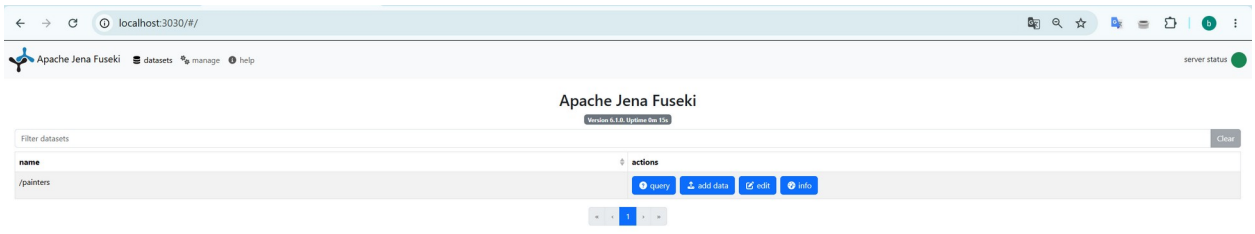


3. Запуск Apache Jena Fuseki

На наступному етапі було виконано перевірку налаштування Apache Jena та Apache Jena Fuseki у середовищі WSL Ubuntu. За допомогою команд echo $JENA_HOME та echo $FUSEKI_HOME було перевірено шляхи до встановлених компонентів. Додатково було виконано перевірку утиліт riot та arq, що підтвердило коректне встановлення Apache Jena версії 6.1.0.

```
user@WIN-6UTALTB07GO:~/laba5$ echo $JENA_HOME
/home/user/laba5/apache-jena-6.1.0
user@WIN-6UTALTB07GO:~/laba5$ echo $FUSEKI_HOME
/home/user/laba5/apache-jena-fuseki-6.1.0
user@WIN-6UTALTB07GO:~/laba5$ riot --version
Apache Jena RIOT version 6.1.0
user@WIN-6UTALTB07GO:~/laba5$ arq --version
Apache Jena version 6.1.0
user@WIN-6UTALTB07GO:~/laba5$ echo $JENA_HOME
/home/user/laba5/apache-jena-6.1.0
user@WIN-6UTALTB07GO:~/laba5$ echo $FUSEKI_HOME
/home/user/laba5/apache-jena-fuseki-6.1.0
user@WIN-6UTALTB07GO:~/laba5$ mkdir -p fuseki-data
user@WIN-6UTALTB07GO:~/laba5$ ss -tulpn | grep 3030 || echo "Порт 3030 вільний"
Порт 3030 вільний
user@WIN-6UTALTB07GO:~/laba5$ fuseki-server --update --loc=./fuseki-data /painters
19:24:13 INFO  Fuseki          :: Apache Jena Fuseki 6.1.0
19:24:14 INFO  Config          :: Fuseki Base = /home/user/laba5/run
19:24:14 INFO  Config          :: Database: /painters
19:24:14 INFO  Config          :: UI Base: fuseki-server.jar
19:24:15 INFO  Shiro           :: Shiro configuration: file:/home/user/laba5/run/shiro.ini
19:24:15 INFO  Server          :: Path = /painters
19:24:15 INFO  Server          :: System
19:24:15 INFO  Server          ::   Memory: 4.0 GiB
19:24:15 INFO  Server          ::   PID:    694
19:24:15 INFO  Server          ::   Port:   http=3030
19:24:15 INFO  Server          :: Start Fuseki
```

Перед запуском сервера було створено директорію fuseki-data, яка використовується для зберігання локальних RDF-даних. Також було перевірено, що порт 3030 є вільним. Після цього сервер Apache Jena Fuseki було запущено командою fuseki-server --update –loc =. /fuseki-data /painters. Параметр --update дозволяє додавати дані до локального dataset, а параметр --loc задає місце зберігання локального triplestore. У результаті було створено dataset /painters, який надалі використовуватиметься як локальний triplestore №1.



Після успішного запуску сервера Apache Jena Fuseki було відкрито веб-інтерфейс за адресою http://localhost:3030. У списку доступних dataset відобразився створений набір даних /painters. Для нього доступні основні дії: виконання SPARQL-запитів через кнопку query, завантаження RDF-даних через add data, редагування через edit та перегляд інформації через info.

Наявність dataset /painters у веб-інтерфейсі підтверджує, що локальний SPARQL endpoint було створено успішно. Саме цей dataset буде використовуватись для зберігання локальних RDF-триплетів і подальшого виконання запитів до локального triplestore.

4. Виконання SPARQL-запиту у Wikidata Query Service

Після запуску локального сервера Apache Jena Fuseki було виконано запит до зовнішнього SPARQL-сервісу Wikidata Query Service. Метою цього етапу була перевірка можливості отримання даних з відкритої графової бази знань через SPARQL. У запиті було відібрано французьких художників, які належать до художнього напряму імпресіонізм. Для цього використовувалися властивості Wikidata, що задають тип сутності, професію, громадянство, художній напрям, дату народження, дату смерті та стать.

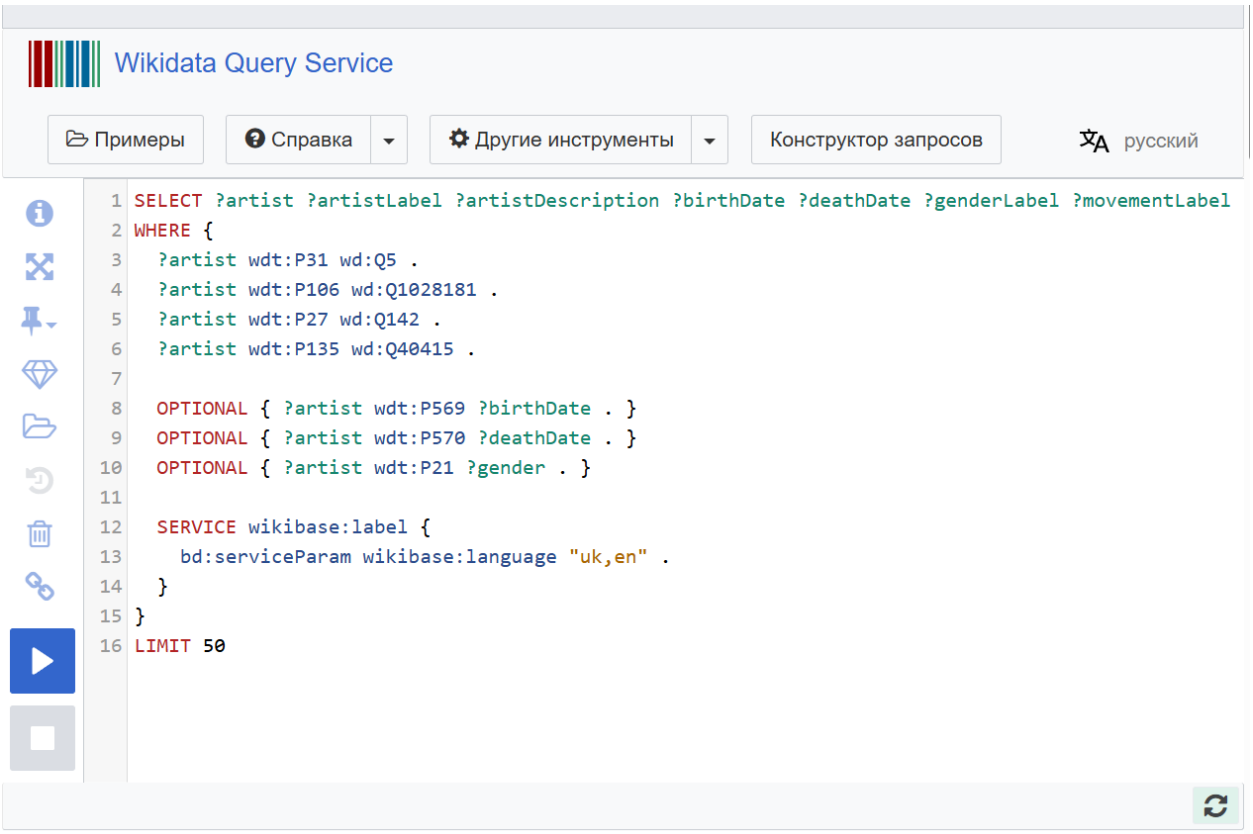


Table ▾ 50 результатов за 568 мс </> Код Скачать ▾ Ссылка ▾

artist	artistLabel	artistDescription	birthDate	deathDate	genderLabel	movementLabel
wd:Q3237916	Pierre Adolphe Valette	French painter (1876-1942)	13 октября 1876 г.	1 января 1942 г.	чоловіча	
wd:Q3270781	Espérance Léon Broquet	French painter (1869-1935)	1 ноября 1869 г.	29 декабря 1935 г.	чоловіча	
wd:Q3371795	Paul Mascart	French painter (1874-1958)	18 апреля 1874 г.	18 ноября 1958 г.	чоловіча	
wd:Q3372389	Paul Vogler	French painter (1852-1904)	1 января 1852 г.	1 декабря 1904 г.	чоловіча	
wd:Q3588288	Émile-Gustave Cavallo-Péduzzi	French painter (1851-1917)	1 января 1851 г.	1 января 1917 г.	чоловіча	
wd:Q4133347	Ганський Петро Павлович	Painter (1867-1942)	1 марта 1867 г.	27 ноября 1942 г.	чоловіча	

У результаті виконання запиту було отримано таблицю з даними про художників. Результат містить ідентифікатор художника, ім’я, опис, дату народження, дату смерті, стать і художній напрям. Це підтверджує, що зовнішній endpoint Wikidata працює коректно і може використовуватися як Internet-endpoint triplestore №2

5. Створення локального RDF-файлу painters.ttl

Далі йдемо до створення локального RDF-файлу і завантаження його у Fuseki. Після перевірки Wikidata створюється файл painters.ttl, у якому описано художників, після чого файл перевіряється через riot і завантажується у dataset /painters через вкладку add data

```
user@WIN-6UTALTB07GO:~/laba5$ cat painters.ttl
@prefix ex: <http://example.org/painters/> .
@prefix dbo: <http://dbpedia.org/ontology/> .
@prefix dbp: <http://dbpedia.org/property/> .
@prefix foaf: <http://xmlns.com/foaf/0.1/> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix wd: <http://www.wikidata.org/entity/> .

ex:Claude_Monet
    a foaf:Person ;
    rdfs:label "Claude Monet"@en ;
    foaf:name "Claude Monet" ;
    dbo:movement "Impressionism" ;
    dbp:wikipedia <https://en.wikipedia.org/wiki/Claude_Monet> ;
    owl:sameAs wd:Q296 .
```

```
user@WIN-6UTALTB07GO:~/laba5$ riot --validate painters.ttl
user@WIN-6UTALTB07GO:~/laba5$ riot --count painters.ttl
painters.ttl    : Triples = 30
user@WIN-6UTALTB07GO:~/laba5$
```

На цьому етапі було створено локальний RDF-файл painters.ttl у форматі Turtle. У файлі описано п’ятьох художників: Claude Monet, Pierre-Auguste Renoir, Camille Pissarro, Paul Cézanne та Edgar Degas. Для кожного художника було задано тип сутності foaf:Person, текстову мітку rdfs:label, ім’я foaf:name, художній напрям dbo:movement, посилання на сторінку Wikipedia через dbp:wikipedia, а також зв’язок із відповідною сутністю Wikidata через предикат owl:sameAs.

Після створення файл було перевірено за допомогою утиліти riot. Команда riot --validate painters.ttl не вивела помилок, що підтверджує коректність синтаксису RDF-файлу. Додатково було виконано команду riot --count painters.ttl, яка показала, що у файлі міститься 30 RDF-триплетів. Це підтверджує, що локальні дані підготовлено коректно і їх можна завантажувати у dataset /painters на сервері Apache Jena Fuseki.

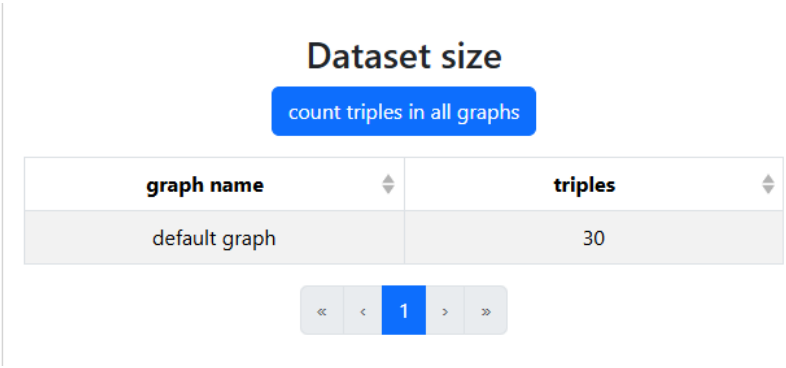
6. Завантаження RDF-файлу у Fuseki

Після створення та перевірки RDF-файлу painters.ttl його було завантажено у локальний dataset /painters за допомогою HTTP-запиту через утиліту curl. Для цього було виконано POST-запит до endpoint http://localhost:3030/painters/data із заголовком Content-Type: text/turtle, що вказує на формат RDF-даних Turtle. У результаті сервер Fuseki повернув відповідь із параметрами count: 30 та tripleCount: 30, що підтверджує успішне додавання 30 RDF-триплетів до локального triplestore.

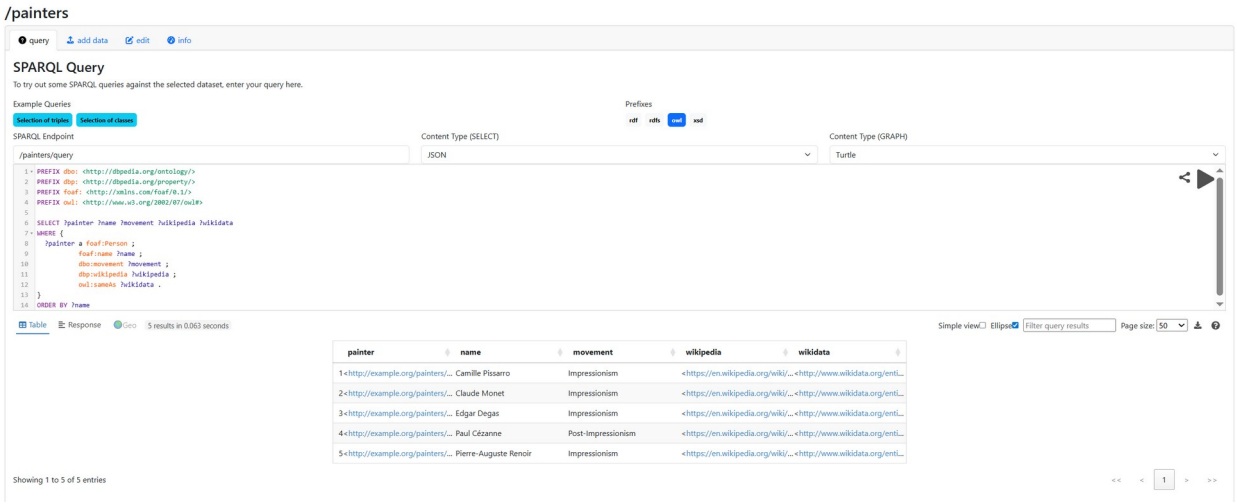
```
user@WIN-6UTALTB07GO:~/laba5$ riot --count painters.ttl
painters.ttl    : Triples = 30
user@WIN-6UTALTB07GO:~/laba5$ curl -X POST \
ntent-Ty>   -H "Content-Type: text/turtle" \
>   --data-binary @painters.ttl \
>   http://localhost:3030/painters/data
{
  "count" : 30 ,
  "tripleCount" : 30 ,
  "quadCount" : 0
}
```

Після завантаження RDF-файлу painters.ttl було перевірено наявність даних у локальному dataset /painters. Сервер Fuseki повернув відповідь із параметрами count: 30 та tripleCount: 30, що підтвердило успішне додавання 30 RDF-триплетів.

```
user@WIN-6UTALTB07GO:~/laba5$ curl -G -H "Accept: text/csv" --data-urlencode 'query=SELECT (COUNT(*) AS ?triples)
 WHERE { ?s ?p ?o }' http://localhost:3030/painters/query
triples
30
```



Додатково у веб-інтерфейсі Fuseki було виконано локальний SPARQL-запит, який зчитує художників, їхні імена, художній напрям, посилання на Wikipedia та відповідні ідентифікатори Wikidata.



Отримання 5 записів підтверджує, що локальний triplestore №1 наповнений даними та готовий до подальшого використання.

7. Об’єднання локальних даних

На цьому етапі було виконано комбінований SPARQL-запит, який поєднує локальні RDF-дані з dataset /painters із зовнішніми даними DBpedia. Локальні дані містили URI художника, ім’я та художній напрям, а додаткова інформація отримувалася з віддаленого SPARQL endpoint https://dbpedia.org/sparql.

Для зв’язування локальних записів із ресурсами DBpedia було використано конструкцію VALUES, у якій кожному локальному імені художника було зіставлено відповідний ресурс DBpedia. Після цього через конструкцію SERVICE було отримано зовнішні дані: англомовну мітку художника, дату народження та дату смерті. У результаті запит повернув 5 записів, що підтверджує успішне об’єднання локальних та інтернет-даних.

Example Queries

Selection of triples | Selection of classes

SPARQL Endpoint

/painters/query

```
PREFIX dbo: <http://dbpedia.org/ontology/>
PREFIX dbp: <http://dbpedia.org/property/>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
PREFIX dbr: <http://dbpedia.org/resource/>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

SELECT ?localPainter ?name ?movement ?dbpediaArtist ?dbpediaLabel ?birthDate ?deathDate
WHERE {
  ?localPainter a foaf:Person ;
                foaf:name ?name ;
                dbo:movement ?movement .

  VALUES (?name ?dbpediaArtist) {
    ("Camille Pissarro" dbr:Camille_Pissarro)
    ("Claude Monet" dbr:Claude_Monet)
    ("Edgar Degas" dbr:Edgar_Degas)
    ("Paul Cézanne" dbr:Paul_Cézanne)
    ("Pierre-Auguste Renoir" dbr:Pierre-Auguste_Renoir)
  }

  SERVICE <https://dbpedia.org/sparql> {
    ?dbpediaArtist rdfs:label ?dbpediaLabel .
    FILTER (lang(?dbpediaLabel) = "en")

    OPTIONAL { ?dbpediaArtist dbo:birthDate ?birthDate . }
    OPTIONAL { ?dbpediaArtist dbo:deathDate ?deathDate . }
```

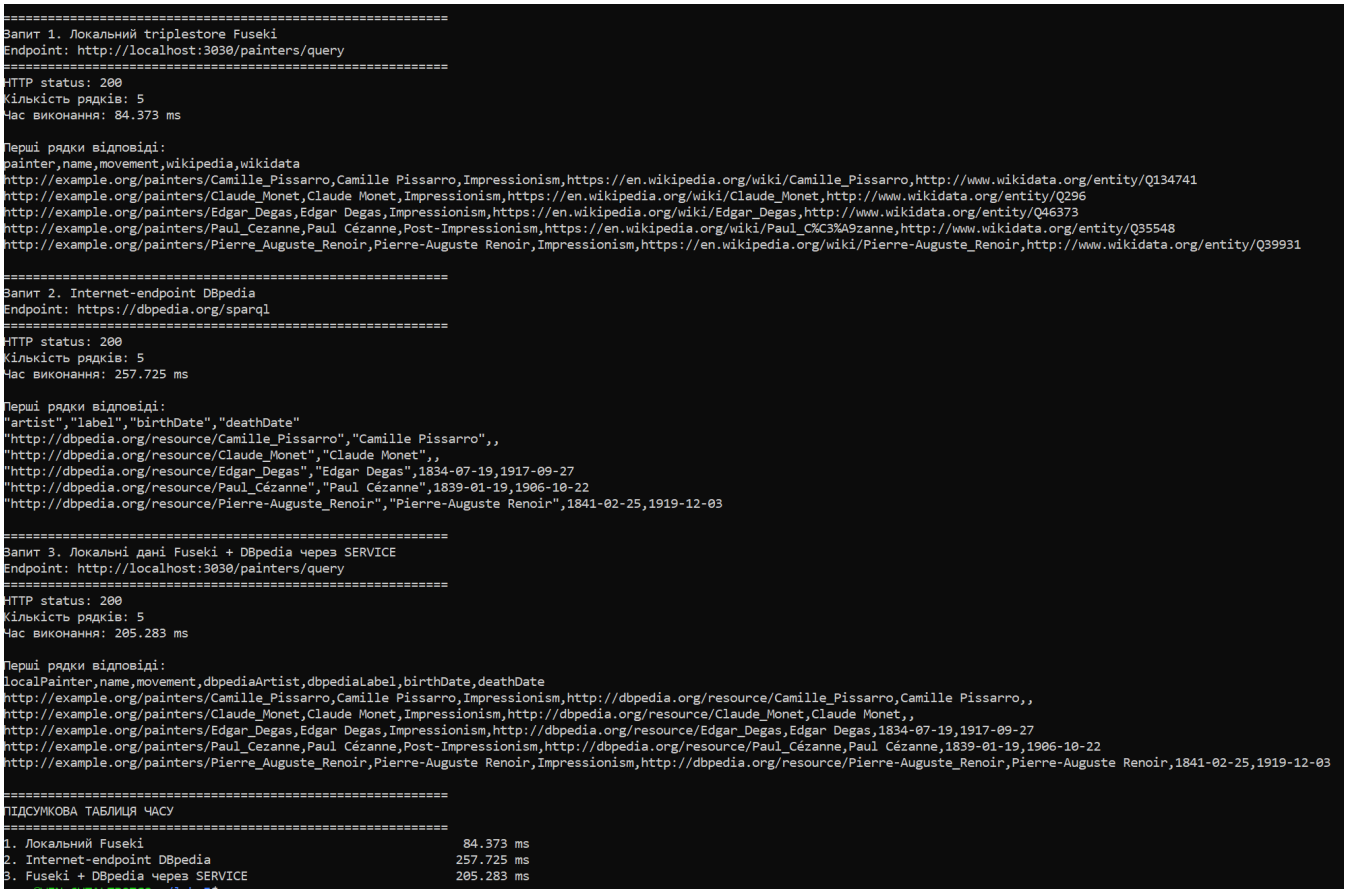
	localPainter	name	movement	dbpediaArtist	dbpediaLabel	birthDate	deathDate
1	<http://example.org/...	Camille Pissarro	Impressionism	<http://dbpedia.org/...	"Camille Pissarro"@en		
2	<http://example.org/...	Claude Monet	Impressionism	<http://dbpedia.org/...	"Claude Monet"@en		
3	<http://example.org/...	Edgar Degas	Impressionism	<http://dbpedia.org/...	"Edgar Degas"@en	"1834-07-19"^^<http:/...	"1917-09-27"^^<http:/...
4	<http://example.org/...	Paul Cézanne	Post-Impressionism	<http://dbpedia.org/...	"Paul Cézanne"@en	"1839-01-19"^^<http:/...	"1906-10-22"^^<http:/...
5	<http://example.org/...	Pierre-Auguste Renoir	Impressionism	<http://dbpedia.org/...	"Pierre-Auguste Reno...	"1841-02-25"^^<http:/...	"1919-12-03"^^<http:/...

Отриманий результат показує, що локальний triplestore №1 /painters може взаємодіяти із зовнішнім triplestore №2 DBpedia. Це підтверджує можливість

роботи з розподіленими графовими даними, коли частина інформації зберігається локально, а частина динамічно отримується з інтернет-джерела.

8. Програмно обчислимо тривалість виклику трьох запитів

Для виконання третього пункту практичного завдання було створено Java-програму FusekiApiTimer.java. Її призначення полягає в тому, щоб програмно виконати три SPARQL-запити та виміряти тривалість кожного з них.



Перший запит був виконаний до локального triplestore Fuseki. Він зчитує дані з dataset /painters, у якому зберігаються RDF-триплети з файлу painters.ttl. У результаті виконання було отримано 5 рядків, що відповідають п’ятьом художникам: Camille Pissarro, Claude Monet, Edgar Degas, Paul Cézanne та Pierre-Auguste Renoir. HTTP-статус 200 підтверджує, що запит був виконаний успішно. Час виконання локального запиту склав 84.373 ms.

Другий запит був виконаний до зовнішнього інтернет endpoint DBpedia. У ньому за допомогою списку конкретних ресурсів DBpedia були зчитані дані про тих самих художників, зокрема англомовна мітка, дата народження та дата смерті. Запит також повернув 5 рядків зі статусом HTTP 200. Час виконання

запиту до DBpedia склав 257.725 ms. Цей запит виконувався довше за локальний, оскільки потребував звернення до зовнішнього сервера через інтернет.

Третій запит був комбінованим. Він виконувався до локального endpoint Fuseki, але всередині запиту використовувалася конструкція SERVICE <https://dbpedia.org/sparql>, яка дозволила отримати додаткові дані з DBpedia. У цьому запиті локальні дані про художників, такі як локальний URI, ім’я та художній напрям, були об’єднані з даними DBpedia, а саме з DBpedia-ресурсом, міткою, датою народження та датою смерті. У результаті було отримано 5 рядків зі статусом HTTP 200. Час виконання комбінованого запиту склав 205.283 ms.

№	Тип запиту	Endpoint	Кількість рядків	Час виконання
1	Локальний triplestore Fuseki	http://localhost:3030/painters/query	5	84.373 ms
2	Internet-endpoint DBpedia	https://dbpedia.org/sparql	5	257.725 ms
3	Fuseki + DBpedia через SERVICE	http://localhost:3030/painters/query	5	205.283 ms

Отримані результати підтверджують, що Java-програма коректно виконує програмне зчитування локального triplestore №1, звернення до інтернет endpoint triplestore №2 та об’єднання локальних і зовнішніх даних.

Висновок

У ході виконання першого практичного завдання було встановлено та налаштовано середовище для роботи з Apache Jena Fuseki. Робота виконувалася у середовищі Ubuntu Linux WSL, де попередньо було перевірено наявність Java JDK 21, Maven та допоміжних утиліт. Після цього було створено робочу директорію laba5, завантажено та розпаковано Apache Jena і Apache Jena Fuseki версії 6.1.0. Також було налаштовано змінні середовища JENA_HOME і FUSEKI_HOME, перевірено роботу утиліт riot та arq, а потім запущено локальний сервер Fuseki командою fuseki-server --update --loc=./fuseki-data /painters. У результаті сервер став доступним через веб-

інтерфейс http://localhost:3030, а dataset /painters було створено як локальний triplestore №1.

Друге практичне завдання було виконано шляхом створення, завантаження та зчитування RDF-даних. Спочатку було створено локальний RDF-файл painters.ttl, у якому описано п’ятьох художників у форматі RDF-триплетів. Файл було перевірено командою riot --validate painters.ttl, а команда riot --count painters.ttl показала наявність 30 триплетів. Далі цей файл було завантажено у Fuseki через HTTP-запит curl до endpoint http://localhost:3030/painters/data, після чого локальний dataset /painters містив 30 RDF-триплетів. Локальний triplestore №1 було зчитано SPARQL-запитом у веб-інтерфейсі Fuseki, де було отримано 5 записів про художників. Internet-endpoint triplestore №2 було перевірено через DBpedia endpoint https://dbpedia.org/sparql, звідки програмно зчитувалися додаткові дані про цих художників. Також було виконано комбінований SPARQL-запит через конструкцію SERVICE, який об’єднав локальні дані з Fuseki з інтернет-даними DBpedia.

Третє практичне завдання було виконано за допомогою Java-програми FusekiApiTimer.java, яка програмно обчислює тривалість виконання трьох запитів. Перший запит звертався до локального endpoint http://localhost:3030/painters/query і зчитував дані з локального triplestore /painters. Другий запит звертався до зовнішнього endpoint DBpedia https://dbpedia.org/sparql і отримував дані з інтернет-бази. Третій запит виконувався до локального Fuseki, але через SERVICE додатково звертався до DBpedia, тобто поєднував локальні та віддалені дані. Усі три запити успішно повернули HTTP status 200 і по 5 рядків результатів. Отриманий час виконання становив: 84.373 ms для локального Fuseki, 257.725 ms для DBpedia та 205.283 ms для комбінованого запиту через SERVICE. Це підтвердило, що всі пункти практичного завдання були виконані: Fuseki встановлено, локальні та інтернет-дані зчитано, їх об’єднання реалізовано, а час виконання трьох запитів обчислено програмно.

Додаток А

```
import java.io.IOException;
import java.net.URI;
import java.net.URLEncoder;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.nio.charset.StandardCharsets;
import java.time.Duration;

public class FusekiApiTimer {
    private static final HttpClient CLIENT = HttpClient.newBuilder()
```

```
            .connectTimeout(Duration.ofSeconds(20))
            .build();

    public static void main(String[] args) {
        String fusekiEndpoint = "http://localhost:3030/painters/query";
        String dbpediaEndpoint = "https://dbpedia.org/sparql";

        String localQuery = """
                PREFIX dbo: <http://dbpedia.org/ontology/>
                PREFIX dbp: <http://dbpedia.org/property/>
                PREFIX foaf: <http://xmlns.com/foaf/0.1/>
                PREFIX owl: <http://www.w3.org/2002/07/owl#>

                SELECT ?painter ?name ?movement ?wikipedia ?wikidata
                WHERE {
                  ?painter a foaf:Person ;
                           foaf:name ?name ;
                           dbo:movement ?movement ;
                           dbp:wikipedia ?wikipedia ;
                           owl:sameAs ?wikidata .
                }
                ORDER BY ?name
                """;

        String dbpediaQuery = """
                PREFIX dbr: <http://dbpedia.org/resource/>
                PREFIX dbo: <http://dbpedia.org/ontology/>
                PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

                SELECT ?artist ?label ?birthDate ?deathDate
                WHERE {
                  VALUES ?artist {
                    dbr:Camille_Pissarro
                    dbr:Claude_Monet
                    dbr:Edgar_Degas
                    dbr:Paul_Cézanne
                    dbr:Pierre-Auguste_Renoir
                  }

                  ?artist rdfs:label ?label .
                  FILTER (lang(?label) = "en")

                  OPTIONAL { ?artist dbo:birthDate ?birthDate . }
                  OPTIONAL { ?artist dbo:deathDate ?deathDate . }
                }
                ORDER BY ?label
                """;

        String combinedQuery = """
                PREFIX dbo: <http://dbpedia.org/ontology/>
                PREFIX foaf: <http://xmlns.com/foaf/0.1/>
                PREFIX dbr: <http://dbpedia.org/resource/>
                PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

                SELECT ?localPainter ?name ?movement ?dbpediaArtist ?
dbpediaLabel ?birthDate ?deathDate
                WHERE {
                  ?localPainter a foaf:Person ;
                                foaf:name ?name ;
                                dbo:movement ?movement .

                  VALUES (?name ?dbpediaArtist) {
                    ("Camille Pissarro" dbr:Camille_Pissarro)
                    ("Claude Monet" dbr:Claude_Monet)
```

```
                    ("Edgar Degas" dbr:Edgar_Degas)
                    ("Paul Cézanne" dbr:Paul_Cézanne)
                    ("Pierre-Auguste Renoir" dbr:Pierre-Auguste_Renoir)
                  }

                  SERVICE <https://dbpedia.org/sparql> {
                    ?dbpediaArtist rdfs:label ?dbpediaLabel .
                    FILTER (lang(?dbpediaLabel) = "en")

                    OPTIONAL { ?dbpediaArtist dbo:birthDate ?birthDate . }
                    OPTIONAL { ?dbpediaArtist dbo:deathDate ?deathDate . }
                  }
                }
                ORDER BY ?name
                """;

        QueryResult localResult = runAndPrint(
                "Запит 1. Локальний triplestore Fuseki",
                fusekiEndpoint,
                localQuery
        );

        pause(2);

        QueryResult dbpediaResult = runAndPrint(
                "Запит 2. Internet-endpoint DBpedia",
                dbpediaEndpoint,
                dbpediaQuery
        );

        pause(2);

        QueryResult combinedResult = runAndPrint(
                "Запит 3. Локальні дані Fuseki + DBpedia через SERVICE",
                fusekiEndpoint,
                combinedQuery
        );

        System.out.println();

System.out.println("============================================================
===");
        System.out.println("ПІДСУМКОВА ТАБЛИЦЯ ЧАСУ");

System.out.println("============================================================
===");
        System.out.printf("%-55s %12.3f ms%n", "1. Локальний Fuseki",
localResult.elapsedMs);
        System.out.printf("%-55s %12.3f ms%n", "2. Internet-endpoint
DBpedia", dbpediaResult.elapsedMs);
        System.out.printf("%-55s %12.3f ms%n", "3. Fuseki + DBpedia через
SERVICE", combinedResult.elapsedMs);
    }

    private static QueryResult runAndPrint(String title, String endpoint,
String query) {
        System.out.println();

System.out.println("============================================================
===");
        System.out.println(title);
        System.out.println("Endpoint: " + endpoint);

System.out.println("============================================================
```

```
===");

        QueryResult result = runQuery(endpoint, query);

        System.out.printf("HTTP status: %d%n", result.statusCode);
        System.out.printf("Кількість рядків: %d%n", result.rows);
        System.out.printf("Час виконання: %.3f ms%n", result.elapsedMs);

        printPreview(result.body, 6);

        return result;
    }

    private static QueryResult runQuery(String endpoint, String query) {
        long start = System.nanoTime();

        try {
            String form = "query=" + URLEncoder.encode(query,
StandardCharsets.UTF_8);

            HttpRequest request = HttpRequest.newBuilder()
                    .uri(URI.create(endpoint))
                    .timeout(Duration.ofSeconds(60))
                    .header("Content-Type", "application/x-www-form-
urlencoded; charset=UTF-8")
                    .header("Accept", "text/csv")
                    .header("User-Agent", "laba5-fuseki-timer/1.0")
                    .POST(HttpRequest.BodyPublishers.ofString(form))
                    .build();

            HttpResponse<String> response = CLIENT.send(request,
HttpResponse.BodyHandlers.ofString());

            long end = System.nanoTime();
            double elapsedMs = (end - start) / 1_000_000.0;
            int rows = countCsvRows(response.body());

            return new QueryResult(response.statusCode(), response.body(),
rows, elapsedMs);

        } catch (IOException | InterruptedException e) {
            long end = System.nanoTime();
            double elapsedMs = (end - start) / 1_000_000.0;

            if (e instanceof InterruptedException) {
                Thread.currentThread().interrupt();
            }

            return new QueryResult(0, "Помилка: " + e.getMessage(), 0,
elapsedMs);
        }
    }

    private static int countCsvRows(String csv) {
        if (csv == null || csv.isBlank()) {
            return 0;
        }

        String[] lines = csv.split("\\R");
        int nonEmpty = 0;

        for (String line : lines) {
            if (!line.isBlank()) {
                nonEmpty++;
```

```
            }
        }

        return Math.max(0, nonEmpty - 1);
    }

    private static void printPreview(String text, int maxLines) {
        System.out.println();
        System.out.println("Перші рядки відповіді:");

        if (text == null || text.isBlank()) {
            System.out.println("(порожня відповідь)");
            return;
        }

        String[] lines = text.split("\\R");

        for (int i = 0; i < Math.min(lines.length, maxLines); i++) {
            System.out.println(lines[i]);
        }

        if (lines.length > maxLines) {
            System.out.println("...");
        }
    }

    private static void pause(int seconds) {
        try {
            Thread.sleep(seconds * 1000L);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }

    private static class QueryResult {
        int statusCode;
        String body;
        int rows;
        double elapsedMs;

        QueryResult(int statusCode, String body, int rows, double elapsedMs)
{
            this.statusCode = statusCode;
            this.body = body;
            this.rows = rows;
            this.elapsedMs = elapsedMs;
        }
    }
}
```